imagine we want to validate that the value of `CurrencyTo` is different from `CurrencyFrom`. Using FluentValidation, you can implement this with an overload of `Must()`, which provides both the model and the property being validated, as shown in the following listing.

Listing 32.14 Using `Must()` to validate that two properties are different

```
RuleFor(x => x.CurrencyTo)                                    ❶
    .NotEmpty()
    .Length(3)
    .MustBeCurrencyCode()
    .Must((InputModel model, string currencyTo)               ❷
        => currencyTo != model.CurrencyFrom)                  ❸
    .WithMessage("Cannot convert currency to itself");        ❹
```

❶ The error message will be associated with the CurrencyTo property.

❷ The Must function passes the top-level model being validated and the current property.

❸ Performs the validation. The currencies must be different.

❹ Uses the provided message as the error message

Creating a validator like this is certainly possible with `DataAnnotations` attributes, but it requires far more ceremony than the FluentValidation equivalent and is generally harder to test. FluentValidation has many more features for making it easier to write and test your validators, too:

- *Complex property validations*—Validators can be applied to complex types as well as to the primitive types like `string` and `int` shown here in this section.
- *Custom property validators*—In addition to simple extension methods, you can create your own property validators for complex validation scenarios.